

Para que los alumnos no se pierdan en el camino, a mí me gusta explicar el ciclo de vida de un proyecto de SQL como una **línea de producción** o la **construcción de un edificio**. Va de lo más abstracto (las ideas del cliente) a lo más concreto (las consultas de datos optimizadas).

Aquí tienes las **6 fases cronológicas** de la creación de un proyecto SQL, explicadas de forma didáctica para tus clases:

Las 6 Fases de un Proyecto SQL

1. Fase de Análisis (El Plano)

- **Qué se hace:** Es la fase que acabamos de trabajar con *notodoescodigo.com*. Hablar con el cliente, entender el negocio, buscar los sustantivos (entidades), sus propiedades (atributos) y cómo se conectan (relaciones).
- **El entregable:** El **Diagrama Entidad-Relación (DER)** o modelo conceptual. Todavía es un dibujo en papel o en herramientas como Draw.io o Lucidchart.

2. Diseño Lógico y Normalización (La Regla)

- **Qué se hace:** Pasamos el dibujo conceptual a tablas reales. Aquí se aplican las **Formas Normales (1FN, 2FN, 3FN)** para evitar la redundancia de datos (que la información no se repita tontamente) y asegurar la integridad.
- **El entregable:** El modelo relacional detallado, donde ya se definen los tipos de datos exactos (**INT**, **VARCHAR**, **DATE**), qué columnas son obligatorias (**NOT NULL**) y cuáles son las claves primarias (**PK**) y foráneas (**FK**).

3. Implementación o DDI (La Construcción)

- **Qué se hace:** ¡Por fin abrimos la consola o el gestor de bases de datos (PostgreSQL, MySQL, SQL Server)! Usamos el lenguaje **DDL (Data Definition Language)** para crear la estructura física en el servidor.
- **El entregable:** El script de creación (**.sql**) lleno de sentencias **CREATE TABLE**, **ALTER TABLE** y las restricciones de integridad (**CONSTRAINTS**).

4. Carga de Datos o DML (La Mudanza)

- **Qué se hace:** Una base de datos vacía no sirve para nada. En esta fase llenamos las tablas con datos. En clase se suele hacer a mano para aprender, pero en el mundo real implica importar archivos CSV, JSON o conectar la base de datos a un proceso ETL/API.
- **El entregable:** Sentencias **DML (Data Manipulation Language)** como **INSERT INTO**. Al acabar esta fase, la base de datos de *notodoescodigo.com* ya tiene usuarios ficticios, cursos creados y regalos en stock.

5. Explotación de Datos o DQL (La Consulta)

- **Qué se hace:** La fase favorita de los analistas. El sistema ya está vivo y el cliente empieza a pedir respuestas. Es el momento de expresar el lenguaje **DQL (Data Query Language)** para transformar datos en conocimiento.
- **El entregable:** Consultas complejas con **SELECT**, filtros con **WHERE**, agrupaciones con **GROUP BY** y uniones de tablas mediante **JOIN**. (Ejemplo: "Saber qué usuario ha canjeado más patitos de goma este mes").

6. Optimización y Mantenimiento (El Ajuste)

- **Qué se hace:** Cuando la plataforma web crece y tiene miles de usuarios, las consultas se vuelven lentas. En esta fase se analiza el rendimiento del servidor, se crean **Índices** para acelerar las búsquedas, se diseñan **Vistas** para simplificar consultas repetitivas y se programan **Triggers** o **Stored Procedures** para automatizar tareas.
- **El entregable:** Una base de datos rápida, segura, con copias de seguridad (backups) programadas y un rendimiento óptimo.



Consejo pedagógico para estructurar tu temario:

A los alumnos les suele desmotivar ver toda la teoría de golpe antes de tocar el teclado. Una buena estrategia es hacer un "**Sprint 0**" **rápido**: les enseñas las fases en una sola clase usando el ejemplo del patito Ducky, montáis una base de datos diminuta de 2 tablas en la primera semana, y luego ya profundizas paso a paso en cada fase con el proyecto grande. ¡Así ven el flujo completo desde el primer momento!

Chicos, imaginen que un cliente les pide construir una casa. ¿Qué es lo primero que hacen? ¿Compran ladrillos y empiezan a apilar de inmediato? No. Primero hablan con el cliente, entienden cuánta gente va a vivir ahí, si quieren piscina o garaje, y dibujan los planos.

En SQL, **la fase de Análisis es el plano de nuestra casa**. No tocamos una sola línea de **CREATE TABLE**. Aquí venimos a hacer preguntas y a entender el negocio."

La fase de análisis se divide en tres pasos que debemos resolver antes de abrir la consola:

1. Identificar las Entidades (¿De qué cosas vamos a guardar datos?)

Lo primero es escuchar al cliente (o leer el requerimiento) y buscar los **sustantivos**. Las entidades son los objetos del mundo real que queremos registrar.

- *Ejemplo:* Si nos piden una base de datos para una tienda, nuestras entidades serán: **Cientes, Productos, Pedidos y Empleados**.

2. Definir los Atributos (¿Qué necesitamos saber de cada cosa?)

Una vez que sabemos qué 'cosas' tenemos, hay que ver qué características nos interesan de ellas. Aquí definimos qué columnas tendrá el futuro proyecto.

- *Para la entidad 'Clientes':* Necesitamos su Nombre, Correo, Teléfono y Dirección. ¿Necesitamos saber su color de ojos? No, no es relevante para el negocio. Analizar también es **filtrar lo que no sirve**.

3. Establecer las Relaciones (¿Cómo se conectan entre sí?)

Las bases de datos relacionales se llaman así por algo. Las cosas no están aisladas. Tenemos que usar la lógica para conectar las entidades:

- Un **Cliente** realiza muchos **Pedidos**.
- Un **Pedido** contiene varios **Productos**.



El "Entregable" de esta fase

Para que los alumnos vean algo concreto, explícales que el resultado final de la fase de análisis no es un archivo **.sql**, sino un **Diagrama Entidad-Relación (DER)**.

Muéstrales un boceto simple en la pizarra: cajitas para las entidades, líneas que las unen para las relaciones, y una lista de atributos. Diles: *"Cuando este dibujo esté aprobado por el cliente y por el equipo, solo entonces, tendremos permiso de escribir nuestro primer **CREATE TABLE**".*



Consejo pedagógico para ti: Para que lo entiendan de verdad, no les des una teoría abstracta. Trae a clase un caso de uso real y caótico (por ejemplo: *"El dueño de una veterinaria me dijo que quiere una app pero no sabe qué datos necesita"*). Haz que ellos te entrevisten a ti (haciendo el papel de cliente) para que experimenten lo difícil y vital que es extraer la información en la fase de análisis.

La Dinámica en Clase: "El Cliente Caótico"

Tu papel: Eres el CEO de *notodoescodigo.com*. Tienes una idea brillante, mucha pasión, pero no sabes nada de bases de datos. Hablas de las funcionalidades mezclándolo todo (puntos, patitos, cursos, perfiles). **El papel de los alumnos:** Son los consultores de datos. Su trabajo es escucharte, frenar tu entusiasmo y estructurar el caos en **Entidades, Atributos y Relaciones**.

Tu discurso para soltarles en clase (El caos inicial):

*"A ver, equipo, quiero montar **notodoescodigo.com**. La idea es que la mascota es un patito de goma (Ducky). Los usuarios entran y pueden tener varios perfiles en su cuenta (uno para su CV web, otro más privado...). Quiero que hagan cursos de programación, que lean el blog y que usen una biblioteca de prompts para IA. También harán tests de softskills para ver si son empáticos o trabajan bien en equipo.*

¡Ah! Y todo es un juego: si completan un curso, ganan 'DuckyPoints'; si aprueban un test, más puntos; si publican un prompt, puntos. Luego, en la tienda del portal, canjean esos puntos por calcetines de Ducky, camisetas de Ducky o un patito gigante. Necesito la base de datos ya, ¿por dónde empezamos?"

La Guía de Resolución (Lo que los alumnos deben descubrir)

Guía a tus alumnos en la pizarra para desgranar el texto anterior en la **Fase de Análisis**.

1. Identificar las Entidades (Sustantivos clave)

Pídeles que identifiquen los "moldes" de información. Deberían llegar a estos:

- **Usuario / Cuenta:** La persona que se registra.
- **Perfil (CV):** El multiperfil que puede tener cada usuario.
- **Curso:** Los contenidos formativos.
- **Artículo (Blog):** Las publicaciones.
- **Prompt:** Los comandos de IA de la biblioteca.
- **Test:** Las pruebas de habilidades blandas.
- **Recompensa (Regalo Ducky):** Los productos de la tienda.
- **Transacción / Historial de Puntos:** El registro de qué puntos gana o gasta cada uno.

2. Definir Atributos (Filtrar y detallar)

Hazles preguntas para definir qué guardamos de cada cosa. Aquí es donde aprenderán a no mezclar.

- ¿El total de puntos va en el Usuario o en el Perfil? (Buen debate para la clase: va en el Usuario, porque los puntos son de la persona, no de un CV específico).
- **Atributos de 'Recompensa':** ID, Nombre del regalo (Ej: "Taza Ducky"), Puntos necesarios, Stock disponible.
- **Atributos de 'Perfil':** ID, Nombre, Titular Profesional, Experiencia laboral, Enlace al GitHub (específicos para el CV).

3. Establecer Relaciones (La magia del "Multiperfil" y los "Puntos")

Aquí los alumnos tendrán que estirar su lógica. Rétales con estas conexiones:

- **Relación Usuario - Perfil (1 a N):** *Un* Usuario puede gestionar *muchos* Perfiles (multiperfil), pero *un* Perfil pertenece a *un solo* Usuario.
- **Relación Usuario - Curso (N a N):** *Un* Usuario puede hacer *muchos* Cursos, y *un* Curso lo pueden hacer *muchos* Usuarios. (Aquí puedes introducir el concepto conceptual de "Matrícula" o "Progreso").
- **El Sistema de Gamificación (La lógica de los DuckyPoints):** ¿Cómo sabe el sistema que cambié puntos por un patito? Necesitamos relacionar al **Usuario** con la **Recompensa**. Cada vez que un usuario canjea algo, se genera un registro con: *ID_Usuario, ID_Recompensa, Puntos_Gastados y Fecha*.



El Momento "¡Ajá!" para cerrar la clase

Para cerrar la sesión de análisis, hazles ver lo siguiente:

"¿Se dan cuenta? El cliente nos habló de diversión, de patitos de goma y de camisetas. Pero nosotros, en la **fase de análisis**, hemos transformado esa idea feliz en una estructura lógica de usuarios, perfiles, contenidos y transacciones.

Si hubiéramos empezado a escribir código SQL desde el primer minuto, nos habríamos olvidado de la tabla de transacciones de puntos, o habríamos metido los cursos dentro del CV del usuario rompiendo el sistema multiperfil. **Analizar nos ha ahorrado semanas de corregir código borrado.**"

Caso Práctico: Diseño de Base de Datos para "notodoescodigo.com"

Asignatura: Bases de Datos / SQL

Fase del Proyecto: Fase 1: Análisis de Requisitos y Modelado Conceptual

Profesor: Óscar

1. Introducción y Contexto

El portal formativo y comunidad de desarrollo **notodoescodigo.com** quiere lanzar una nueva plataforma web revolucionaria. El objetivo de este sitio es permitir a los usuarios centralizar su marca profesional, formarse, medir sus habilidades y, además, divertirse en el proceso a través de un sistema de gamificación guiado por **Ducky**, la mascota oficial del portal (un carismático patito de goma).

Como consultores de datos, se nos ha encomendado la tarea de **diseñar la arquitectura de la base de datos** que dará soporte a todo este ecosistema.

2. Requerimientos del Sistema (Briefing del Cliente)

Tras las reuniones iniciales con el equipo de *notodoescodigo.com*, se han extraído las siguientes necesidades y reglas de negocio que el sistema debe cumplir:

A. Gestión de Usuarios y Multiperfil

- La plataforma contará con **Usuarios** registrados. De cada usuario se necesita almacenar un identificador único, su nombre, apellidos, correo electrónico, contraseña (encriptada) y la fecha de registro.
- Un usuario debe poder gestionar **múltiples perfiles (CV web)** dentro de su misma cuenta (por ejemplo: un perfil enfocado a *Backend Developer*, otro para *Fullstack*, y otro privado).
- Cada **Perfil** tendrá su propio título profesional, una biografía, experiencia laboral, habilidades técnicas y enlaces a redes (como GitHub o LinkedIn).

B. Contenido y Recursos

El portal ofrece cuatro grandes secciones de interacción:

1. **Cursos:** Formaciones técnicas. Cada curso tiene un título, descripción, horas de duración y un nivel (Básico, Intermedio, Avanzado).
2. **Blog:** Artículos de divulgación. De cada artículo se registra el título, el contenido, la fecha de publicación y el autor.

3. **Biblioteca de Prompts:** Un repositorio de comandos de Inteligencia Artificial. Cada prompt almacenado tendrá el comando de texto exacto, la tecnología a la que va dirigido (ej: *ChatGPT*, *Midjourney*) y una breve explicación de su utilidad.
4. **Tests de Soft Skills:** Pruebas de opción múltiple para evaluar habilidades blandas (empatía, liderazgo, trabajo en equipo). Cada test tiene un nombre de la habilidad a evaluar y un número de preguntas.

C. Gamificación y la "Tienda Ducky"

Para fomentar la participación, todo el portal está gamificado mediante **DuckyPoints**:

- Los usuarios acumulan puntos en su cuenta global cada vez que realizan una acción: completar un curso, leer un artículo, publicar un prompt o aprobar un test de soft skills.
- Existe un catálogo de **Recompensas** (merchandising temático de Ducky). Cada producto de la tienda tiene un nombre (ej: *Camiseta Ducky*, *Calcetines informáticos*, *Patito de goma gigante*), una descripción, el stock disponible y el coste en puntos necesarios para conseguirlo.
- Los usuarios pueden **intercambiar sus puntos** por estos regalos. El sistema debe dejar constancia exacta de qué usuario ha canjeado qué regalo, cuántos puntos le ha costado y en qué fecha se realizó el intercambio.

3. Objetivos de la Práctica (Lo que debes entregar)

Recuerda la regla de oro: **No se abre el gestor de SQL hasta que el plano esté terminado**. Para dar por finalizada esta fase de Análisis, debes presentar los siguientes tres puntos:

1. **Listado de Entidades:** Identifica los moldes principales de información que requiere el sistema.
2. **Diccionario de Atributos:** Para cada entidad, define qué datos específicos (columnas) se deben almacenar y debate en grupo cuál debería ser la "Clave Primaria" (Primary Key) que identifique unívocamente cada registro.
3. **Boceto de Relaciones:** Explica con tus palabras o mediante un dibujo simple cómo se conectan las entidades entre sí (si las relaciones son de 1 a N, o de N a N). Pon especial atención a cómo conectar al usuario con el sistema de multiperfil y con el canje de puntos por regalos.

💡 “Si no puedes dibujarlo en un papel, no sabrás programarlo en SQL.” ¡A analizar!

Mapa de Relaciones (La Lógica del DER)

Antes de pintar las tablas con todas sus columnas, los alumnos deben entender cómo se conectan los bloques. Puedes dibujar este mapa de dependencias en la pizarra:

- **Usuarios** —(1:N)—> **Perfiles**: Un usuario crea muchos perfiles (CVs), pero cada perfil pertenece a un único usuario.
- **Usuarios** —(1:N)—> **Prompts**: Un usuario puede publicar muchos prompts en la biblioteca.
- **Usuarios** —(N:N)—> **Cursos**: Muchos usuarios hacen muchos cursos. *Genera tabla intermedia: **Progresos_Cursos**.*
- **Usuarios** —(N:N)—> **Tests**: Muchos usuarios realizan muchos tests de softskills. *Genera tabla intermedia: **Resultados_Tests**.*
- **Usuarios** —(N:N)—> **Recompensas**: Muchos usuarios canjean muchos regalos de Ducky. *Genera tabla intermedia: **Canjes_Regalos** (la Tienda).*

Estructura del DER (Tablas, Atributos y Claves)

Aquí tienes el diseño detallado del modelo relacional. Las líneas que conectan las tablas se definen mediante las **Claves Foráneas (FK)**, que apuntan siempre a la **Clave Primaria (PK)** de la tabla origen.

Bloque de Identidad y Marca

Tabla: **Usuarios**

Guarda la cuenta global y el saldo de puntos acumulados de la persona.

- **id_usuario** (**PK**, INT, Auto_increment)
- **nombre** (VARCHAR)
- **apellidos** (VARCHAR)
- **email** (VARCHAR, Unique)
- **password_hash** (VARCHAR)
- **duckypoints_actuales** (INT) — *Saldo de puntos disponibles para gastar.*
- **fecha_registro** (DATE)

Tabla: **Perfiles**

Los diferentes CVs web que un usuario puede crear (Multiperfil).

- **id_perfil** (**PK**, INT, Auto_increment)
- **id_usuario** (**FK** apuntando a **Usuarios.id_usuario**) — *Saber de quién es este CV.*
- **titulo_profesional** (VARCHAR) — *Ej: "Fullstack Developer"*
- **biografia** (TEXT)
- **experiencia_laboral** (TEXT)

- `linkedin_url` (VARCHAR)

Bloque de Contenido e Interacción

Tabla: `Cursos`

- `id_curso` (**PK**, INT, Auto_increment)
- `titulo` (VARCHAR)
- `descripcion` (TEXT)
- `horas_duracion` (INT)
- `nivel` (VARCHAR) — *Básico, Intermedio, Avanzado*
- `puntos_recompensa` (INT) — *Cuántos DuckyPoints da al terminarlo.*

Tabla: `Prompts`

- `id_prompt` (**PK**, INT, Auto_increment)
- `id_usuario` (**FK** apuntando a `Usuarios.id_usuario`) — *El autor que lo compartió.*
- `comando_texto` (TEXT)
- `tecnologia` (VARCHAR) — *Ej: "ChatGPT", "Midjourney"*
- `explicacion` (TEXT)

Tabla: `Tests_Softskills`

- `id_test` (**PK**, INT, Auto_increment)
- `habilidad_evaluada` (VARCHAR) — *Ej: "Empatía", "Liderazgo"*
- `num_preguntas` (INT)
- `puntos_recompensa` (INT)

Bloque de Gamificación e Historial (Tablas Intermedias)

Aquí es donde se produce la magia de la base de datos relacional. Estas tablas rompen las relaciones Muchos a Muchos (N:N) y registran las acciones del usuario.

Tabla: `Progresos_Cursos` (Relación Usuario-Curso)

Registra qué cursos está haciendo o ha terminado cada usuario.

- `id_progreso` (**PK**, INT, Auto_increment)
- `id_usuario` (**FK** apuntando a `Usuarios.id_usuario`)
- `id_curso` (**FK** apuntando a `Cursos.id_curso`)
- `estado` (VARCHAR) — *'En curso', 'Completado'*
- `fecha_completado` (DATE, Nullable)

Tabla: `Resultados_Tests` (Relación Usuario-Test)

Registra la puntuación de softskills obtenida.

- `id_resultado` (**PK**, INT, Auto_increment)
- `id_usuario` (**FK** apuntando a `Usuarios.id_usuario`)
- `id_test` (**FK** apuntando a `Tests_Softskills.id_test`)
- `puntuacion_obtenida` (INT)
- `fecha_intento` (DATE)

Tabla: **Recompensas** (La Tienda Ducky)

El catálogo de regalos disponibles.

- `id_recompensa` (**PK**, INT, Auto_increment)
- `nombre_regalo` (VARCHAR) — Ej: "Camiseta Ducky"
- `descripcion` (TEXT)
- `coste_puntos` (INT) — *Cuántos puntos vale.*
- `stock` (INT)

Tabla: **Canjes_Regalos** (Relación Usuario-Recompensa)

El historial de compras de la tienda. Crucial para saber a quién enviarle los calcetines de Ducky.

- `id_canje` (**PK**, INT, Auto_increment)
- `id_usuario` (**FK** apuntando a `Usuarios.id_usuario`)
- `id_recompensa` (**FK** apuntando a `Recompensas.id_recompensa`)
- `puntos_gastados` (INT) — *Se guarda el histórico por si el precio del regalo cambia en el futuro.*
- `fecha_canje` (DATE)



Dinámica para la pizarra con tus alumnos:

Cuando pintes esto en clase, hazles ver el **flujo del dinero (puntos)**:

1. Un usuario completa un registro en `Progresos_Cursos` con estado 'Completado'.
2. El sistema mira los `puntos_recompensa` de ese curso y se los suma a los `duckypoints_actuales` de la tabla `Usuarios`.
3. Más tarde, el usuario va a la tienda, elige una `Recompensa` y se crea una fila en `Canjes_Regalos`.
4. El sistema resta esos puntos del saldo del usuario.

¡Con este diseño tienen un mapa perfecto para saltar a la Fase 3 y empezar a escribir el código de creación de tablas!